

FT_VECTOR_SCRIPT_ENGINE (COMPLETE SPECIFICATION)

An Atomic, Deeply Flattened, Zero-Step-Spike Technical Curriculum

Syllabus Code:	RUST_VGE_ATOMIC_ULTIMATE_REF	Level:	Systems Software & Applied Compilers
Allowed Language:	Rust (Stable Edition)	Norm Requirements:	Absolute error handling. No unhandled panics or unwraps.

MOLINETTE VERIFICATION & TECHNICAL GRADING NORMS

- Atomic Verification Checkpoints:** Every sub-sub-step (X.Y.O.Z) must compile as an independent testable unit with clean assertion validations.
- Dual-Purpose Vector Semantics:** SVG assets must act simultaneously as graphic render sheets AND structured algebraic spatial collision/trigger boundaries.
- Zero-Panic Paradigm:** The use of `.unwrap()` or `.expect()` is strictly prohibited. Programs must fallback gracefully using `match`, `Result`, and `Option` layout variations.

PROJECT 00: FT_QUEST_MANAGER (DATA AND I/O FOUNDATIONS)

Isolate basic memory bounds, ownership transfers, text sequence parsing, and clean JSON persistence layers before scaling into vector geometry subsystems.

Turn-in directory: `ex00/` Dependencies: `serde, serde_json`

Milestone 0.0 – Allocation Layout Models

Sub-step 0.0.0.1: Plain Type Structural Layouts

Construct a basic `Quest` struct managing only plain primitive variables: `id: u32` and `is_done: bool`. Instantiate this structure cleanly in your main loop block to pass initial compilation checks.

- Expected Output Example:** Internal compile check: `cargo check` returns status 0 with no syntax structural alerts.
- Good Practice Tip:** Keep structure fields private by default unless they explicitly need exposure across wider modular horizons.
- Syntax Philosophy Warning:** In Rust, structures without explicit implementation methods behave like pure data records. Avoid declaring unmanaged global static metrics.

Sub-step 0.0.0.2: Heap String Binding Integration

Expand the `Quest` block by appending dynamic, heap-allocated properties: `title: String` and `desc: String`. Practice setting properties using string initialization routines like `String::from()` or `.to_string()`.

- Expected Output Example:** `Quest { id: 1, title: "Piscine", desc: "Start", is_done: false }` via macro debugging.
- Good Practice Tip:** Never store static text views directly inside operational memory entities; enforce explicit allocations.
- Syntax Philosophy Warning:** Plain string literals (`&str`) represent static, immutable views pointing straight to the compiled binary block. Storing them inside structures forces complex lifetime tracking descriptors (`<'a>`). Use heap-allocated, owned `String` data structures to simplify memory layout models.

Sub-step 0.0.0.3: Heap-Contiguous Array Containers

Create a separate `QuestLog` wrapper struct containing a growable, heap-allocated collection vector: `Vec<Quest>`. Write an instantiation helper function that creates an empty layout container sequence on initialization.

- Expected Output Example:** Container structures trace: `QuestLog { list: [] }`
- Good Practice Tip:** Implement standard constructor conventions using a clean `pub fn new() -> Self` implementation on all custom wrappers.
- Syntax Philosophy Warning:** Vectors manage dynamically resizable, contiguous blocks of tracking bytes on the heap. When the parent structural container drops out of operational scope limits, Rust drops and frees all child items sequentially.

Milestone 0.1 – Ownership Rules & Mutability

Sub-step 0.1.0.1: Exclusive Mutable Borrows

Implement an explicit data entry method on your wrapper layout: `pub fn add_quest(&mut self, quest: Quest)`. Add element nodes securely using an exclusive mutable borrow configuration.

Expected Output Example: Trace confirming operation: "Added quest node: Piscine Start" printed directly to stdout.

Good Practice Tip: Keep active borrowing scopes as narrow as possible. Do not hold exclusive reference hooks across long, unmanaged code blocks.

Syntax Philosophy Warning: The compiler enforces a strict access design pattern: you can maintain an infinite count of shared immutable views (`&T`) OR exactly one exclusive mutable loan (`&mut T`) inside any single block scope, removing data races.

Sub-step 0.1.0.2: Tracking Memory Moves

Write an intentionally broken verification statement in `main()` that attempts to read the contents of a `Quest` instance immediately after passing it into the `add_quest()` routine. Observe the compiler check behavior.

Expected Output Example: Expected compiler block error: `error[E0382]: use of moved value: 'quest'`

Good Practice Tip: Intentionally trigger this lifecycle behavior during peer evaluations to show how the borrow checker blocks invalid code before runtime.

Syntax Philosophy Warning: Passing a non-scalar object into an external function context shifts its ownership tracking registry entirely. The calling scope completely loses validation permissions over that item. To keep access, pass clean shared borrows (`&`) or clone the instance.

Milestone 0.2 – Static Inspection Loop Processing

Sub-step 0.2.0.1: Shared Structural Iteration

Write an operational read-only logging method: `pub fn print_all(&self)`. Use an explicit `for` reference loop to inspect vector items without taking ownership out of the array container.

Expected Output Example: Printed line readout: `[ID: 1] Piscine Start (Status: Pending)`

Good Practice Tip: Enforce loop iterations directly over shared collection vectors (`for quest in &self.list`) to maintain continuous data storage.

Syntax Philosophy Warning: Iterating over raw, unreferenced collections moves elements directly out of the tracking layout, draining the parent vector framework entirely.

Sub-step 0.2.0.2: Structural Pattern Filtering

Write an isolated filtering readout function: `pub fn print_completed(&self)`. Inside your loop pass, apply structural pattern validation through clean conditional matching like `if let true = quest.is_done`.

Expected Output Example: `-- Completed Records Only -- [ID: 2] SVG Logic (Status: Complete)`

Good Practice Tip: Use concise matching rules to simplify your layout checks when querying configuration flags.

Syntax Philosophy Warning: Pattern matching syntax across Rust models requires complete coverage of all logical paths, forcing structural integrity onto your variables.

Milestone 0.3 – Text Cleaning and Mapping Mechanics

Sub-step 0.3.0.1: Stripping Newline Markers

Write a text sanitization function that inputs a raw string slice (`&str`) containing whitespace or trailing control signals (`\n` or `\r`) and applies `.trim()` to isolate valid content.

Expected Output Example: Raw dirty data input: `" 42 "` yields clean string slice: `"42"`.

Good Practice Tip: Always clean raw data strings before passing them to parsing routines to strip out invisible text metadata introduced by user terminal shells.

Syntax Philosophy Warning: Rust string objects are strictly guaranteed to represent valid, clean UTF-8 text characters. Slicing raw strings by indexing byte bounds can cause execution crashes if multi-byte sequences are split. Use standard text utility cleaners.

Sub-step 0.3.0.2: Safe Numeric Translation

Pass your sanitized string slice to `.parse::()`. Wrap the final operation result inside an explicit type-safe `Result<u32, &'static str>` envelope to manage conversion processing securely without crashes.

Expected Output Example: Action outcomes: `Ok(42)` or an error envelope: `Err("Parsing phase failed: Invalid numeric notation")`.

Good Practice Tip: Map exceptions cleanly using native core enums early, preventing unexpected error propagation down your call stack.

Syntax Philosophy Warning: Avoid calling helper methods like `.unwrap()`. It triggers a fatal panic abort sequence if conversion validation fails. Propagate your parsing anomalies using the standard `?` modifier block.

Milestone 0.4 – Shell Infinite Loop Execution

Sub-step 0.4.0.1: Stream Buffering Mechanics

Construct a stable non-terminating framework execution `loop {}`. Instantiate a persistent tracking buffer text allocation and populate string inputs securely via `std::io::stdin().read_line(&mut buffer)`.

Expected Output Example: Active interface view prompt: `vge_shell> _` waiting continuously for command input.

Good Practice Tip: Flush and empty your tracking buffer string configuration via `.clear()` at the start of every loop path to keep previous entry remnants from polluting active passes.

Syntax Philosophy Warning: Standard input tracking sequences block thread processing cleanly, sleeping the program until interactive bytes break across the stream threshold.

Sub-step 0.4.0.2: Match-Driven Shell Routing

Pipe input tracking buffers into your module 0.3 utility parser. Use a `match` routing layout: process verified codes to complete records, or report anomalies without interrupting the overarching loop.

Expected Output Example: User alerts output: `Error: Input must represent a valid identification number. Re-enter text. vge_shell>`

Good Practice Tip: Always construct explicit exit flags (like matching strings `"exit"` or `"quit"`) to drop memory allocations cleanly on shutdown.

Syntax Philosophy Warning: Encapsulating infinite iteration frames directly inside pattern-matching routers forms the base architectural scaffolding for type-safe engine loop layers.

Milestone 0.5 – Filesystem Persistence Layers

Sub-step 0.5.0.1: JSON Serialization Output

Decorate structures with macro serialization flags. Export your data arrays directly into flat file configurations on disk as `quests.json` via standard helper operations like `std::fs::write()`.

Expected Output Example: Flat filesystem code file content: `[{"id": 1, "title": "Piscine", "desc": "Start", "is_done": false}]`

Good Practice Tip: Format target output serialization traces into clean human-readable layouts to keep historical debugging paths clear.

Syntax Philosophy Warning: Serialization algorithms flatten complex heap-allocated structure layouts down into linear sequences of characters.

Sub-step 0.5.0.2: JSON Ingestion Error Recovery

Implement file ingestion tracking using `std::fs::read_to_string()`. If files are missing or corrupt, handle the exception cleanly and fallback to initializing default empty structures.

Expected Output Example: Diagnostic alert: `Warning: 'quests.json' inaccessible. Spawning clean collection matrix context...`

Good Practice Tip: Map missing target records as standard execution branches instead of raising critical exceptions or letting the thread fail.

Syntax Philosophy Warning: File I/O checks return standard `Result<String, std::io::Error>` configurations, forcing you to handle error paths explicitly before accessing data.

PROJECT 01: FT_POLYMORPHIC_NODES (THE CONTINUOUS STRUCTURAL GRAPH TREE)

Deconstruct Object-Oriented inheritance patterns into type-safe data composition models. Isolate World Space coordinates from user interface components using structural enums.

Turn-in directory: ex01/

Dependencies: std::*

Milestone 1.0 – Component Structural Data Layouts

Sub-step 1.0.0.1: Character Payload Records

Define an isolated data record block named `CharacterData` managing moving engine entities: `hp: i32`, `max_speed: f32`, and `radius: f32`.

Expected Output Example: Internal compiler check: Structural parameters compile successfully with continuous floating point trackers.

Good Practice Tip: Enforce type-specific float variables across physics dimensions to keep spatial updates highly precise.

Syntax Philosophy Warning: The compiler never performs implicit numeric type promotions. Combining integers and floats requires explicit conversion using the `as` keyword.

Sub-step 1.0.0.2: Terrain & Interactive Object Blocks

Create two explicit static property record sheets: `TerrainData { layer_id: u32 }` to direct rendering sorting, and `ObjectData { is_trigger_active: bool }` for checking level elements.

Expected Output Example: Internal compile verification: Separate structure containers load and register cleanly.

Good Practice Tip: Keep configuration keys distinct from live runtime tracking statistics to maintain clean structural records.

Syntax Philosophy Warning: Grouping operational records cleanly allows the compiler to optimize data spacing, reducing cash-miss delays during execution updates.

Milestone 1.1 – Level 2 Category Division Enums

Sub-step 1.1.0.1: The In-World Payload Enum

Create an `InWorldType` enum layout. Configure its variants to wrap your component records directly as concrete data payloads: `Character(CharacterData)`, `Terrain(TerrainData)`, and `Object(ObjectData)`.

Expected Output Example: Trace validation check: `InWorldType::Character(CharacterData { hp: 100, ... })` evaluates cleanly.

Good Practice Tip: Keep data variant configurations decoupled so you can reuse specific structures across wider layout contexts.

Syntax Philosophy Warning: Enum variants can carry concrete payloads directly, combining type identification flags and raw data fields into a single allocation block.

Sub-step 1.1.0.2: The UI Overlay Interface Enum

Build an independent `UiType` enum that maps screen space layout interfaces. Provide structural parameters to track widgets: `Button { active_color: u32 }` and `TextBox { text_content: String }`.

Expected Output Example: Model debug trace: `UiType::Button { active_color: 0xFF0000 }` prints successfully.

Good Practice Tip: Enforce distinct positioning guidelines for user interfaces to keep rendering updates highly organized.

Syntax Philosophy Warning: Separating UI properties from physical spatial parameters prevents system updates from evaluating invalid calculations or mixing up layout scopes.

Milestone 1.2 – Level 1 Polymorphic Tree Merging

Sub-step 1.2.0.1: Master Category Merging

Synthesize separate subclass models into a single top-level master classification tree enum: `pub enum NodeCategory { InWorld(InWorldType), UserInterface(UiType) }`.

Expected Output Example: Internal compiler check: Complex nested algebraic trees evaluate and resolve successfully.

Good Practice Tip: Map structural hierarchies clearly using nested enums to simplify type-checking loops for your peers.

Syntax Philosophy Warning: Enums provide safe type polymorphism without introducing performance penalties or relying on bloated dynamic vtable dispatch trackers.

Sub-step 1.2.0.2: Unified Header Packaging

Create the master `GameNode` structural header matching properties common across all entities: `id: usize`, `name: String`, continuous coordinates (`position_x: f32`, `position_y: f32`), and your variant block: `behavior: NodeCategory`.

Expected Output Example: Node object snapshot: `GameNode { id: 0, name: "Player_1", position_x: 45.2, behavior: ... }`

Good Practice Tip: Store uniform metadata properties directly in the master wrapper header to avoid repetitive lookup operations across your sub-variants.

Syntax Philosophy Warning: Grouping shared parameters inside a single header block allows engine filters to process coordinates and transforms quickly without parsing inner records.

Milestone 1.3 – The Pointerless Arena Collection Tree

Sub-step 1.3.0.1: Continuous Vector Allocation Matrices

Construct a `SceneTree` structure tracking a centralized linear allocation matrix: `Vec<GameNode>`. You are strictly forbidden from utilizing reference-counting pointers like `Rc` or `Arc` to link system graphs.

Expected Output Example: Master container layout: `SceneTree { nodes: [GameNode, GameNode] }`

Good Practice Tip: Use an arena data-allocation pattern to manage nested graphs easily without getting blocked by borrow checker restrictions.

Syntax Philosophy Warning: Using traditional heap reference loops can easily form pointer cycles, causing memory leaks. Storing nodes in flat arrays allows you to clean up data life cycles reliably without leaking memory.

Sub-step 1.3.0.2: Index ID Allocation Trackers

Write an operational database entry routine that adds records and yields their flat array index offset (`usize`). Use these numerical index codes to link parent-child node connections across your collection.

Expected Output Example: Graph link registration view: `"Linked parent index node: 0 -> Child entity index: 1"`

Good Practice Tip: Enforce primitive index tracking codes instead of reference pointers to define structural connections across your graph models.

Syntax Philosophy Warning: An index code (`usize`) safely references an entry in an allocation block without pinning down the entire collection under a continuous, restrictive borrow lock.

Milestone 1.4 – Nested Pattern Downcasting Passes

Sub-step 1.4.0.1: Deep Structural Match Routing

Write a localized diagnostic screening function: `fn audit_character_health(tree: &SceneTree)`. Query items via reference structures and apply deep nested matching to locate and extract HP records from inner characters.

Expected Output Example: Log output readout: `"Identified Character Entity: 'Hero_Knight' | HP Metric: 85"`

Good Practice Tip: Use clean multi-level nested matching paths to unwrap specific attributes from deep within your structural options.

Syntax Philosophy Warning: Deep pattern matching safely processes target variants at compile time without relying on risky or expensive runtime type conversions.

Sub-step 1.4.0.2: Explicit Variant Bypassing

Ensure your diagnostic pass includes an explicit fallback arm (`_ => {}`) to safely skip mismatched categories like UI objects or static scenery elements without compile-time tracking faults.

Expected Output Example: UI containers and static assets are skipped smoothly during diagnostic calculation loops.

Good Practice Tip: Always provide structural fallback branches to handle unselected categories explicitly.

Syntax Philosophy Warning: Exhaustive pattern matching forces you to account for all possible system variants, removing unexpected logic cracks from your codebase.

PROJECT 02: FT_VECTOR_PHYSICS (CONTINUOUS SVG SPACES)

Deprecate grid tracking constraints completely. Parse boundary layouts directly out of raw SVG level sheets and calculate coordinate checks using continuous floating-point mathematics.

Turn-in directory: ex02/ **Dependencies:** `roxmltree`, `lyon_geom` (or structural segment line calculations)

Milestone 2.0 – Vector Polygon Layout Specifications

Sub-step 2.0.0.1: Mathematical Vertex Storage Arrays

Create a `Polygon2D` struct containing a continuous vector list tracking coordinate bounds: `vertices: Vec<(f32, f32)>`. Configure a static four-point square boundary outline manually inside your validation scope.

Expected Output Example: Geometry debug trace: `Polygon2D { vertices: [(0.0, 0.0), (100.0, 0.0), (100.0, 100.0), (0.0, 100.0)] }`

Good Practice Tip: Verify that your vector point coordinates close completely by matching your initial vertex parameters to your terminal bounds.

Syntax Philosophy Warning: Floating-point representations can introduce minor tracking rounding variations. Avoid checking float values for exact equality (`==`); use threshold tolerance values instead.

Sub-step 2.0.0.2: Ray-Casting Intersect Algorithms

Implement a ray-casting boundary evaluation technique: `pub fn contains_point(&self, px: f32, py: f32) -> bool`. Track ray segment edge crossings to mathematically verify coordinate placement.

Expected Output Example: Math query outcome: `"Coordinate Point evaluation (50.0, 50.0) inside bounds: true"`

Good Practice Tip: Handle edge variations safely during cross calculations to protect against division-by-zero errors.

Syntax Philosophy Warning: Encapsulating mathematical parameters within pure, stateless logic functions simplifies code optimization and ensures highly reliable testing.

Milestone 2.1 – Proximity Line Calculations

Sub-step 2.1.0.1: Proximity Vector Checking Math

Write an explicit vector calculation utility that determines the shortest spacing between an entity position (cx, cy) and a linear line segment running from coordinate vertices (x_1, y_1) to (x_2, y_2) .

Expected Output Example: Distance calculation readout: `"Shortest vector spacing to wall line segment: 12.45 units"`

Good Practice Tip: Break complex geometry computations into small, independent sub-calculations to make troubleshooting layout models simple.

Syntax Philosophy Warning: Breaking geometry passes into specialized routines lets the compiler apply aggressive vector optimization paths across your code loops.

Sub-step 2.1.0.2: Radius-Bound Intersection Tests

Write a circle proximity check on your shapes: `pub fn intersects_circle(&self, cx: f32, cy: f32, r: f32) -> bool`. Check wall spacing against character radius limits to catch collisions before entries slide through lines.

Expected Output Example: Safety alert trace: `"Collision detected! Character radius boundary intersects wall segment."`

Good Practice Tip: Evaluate proximity checks directly against an entity's bounding radius to prevent objects from clipping through thin line barriers.

Syntax Philosophy Warning: Catching coordinate changes early gives you full control over position updates before committing final modifications to core memory nodes.

Milestone 2.2 – Asset Metadata Extraction (XML Parsing)

Sub-step 2.2.0.1: XML Layer Filter Extensions

Use `roxmltree` to load an external vector layout sheet. Write a search operation that walks the tree, isolates a specific layer ID like `<g id="collision_walls">`, and reads its path data while ignoring graphics layers.

Expected Output Example: Extracted string verification: `"Located wall text layer: M 10 20 L 50 80 C 100 ..."`

Good Practice Tip: Enforce strict naming rules across your vector layers to keep XML filtering sweeps clean and predictable.

Syntax Philosophy Warning: Tree parsing methods yield zero-copy string slice references tied directly to the source asset's lifetime, preventing unnecessary string duplication.

Sub-step 2.2.0.2: SVG Path Command Interpreters

Parse the raw text inside the path's `d="..."` parameter. Construct a scan loop that evaluates path signals: convert **M** (Move To) and **L** (Line To) strings into clean numeric arrays.

Expected Output Example: Output token list: `[MoveTo(10.0, 20.0), LineTo(50.0, 80.0)]` parsed from text codes.

Good Practice Tip: Parse parameters systematically using a clear parsing loop, capturing and reporting file format alerts cleanly.

Syntax Philosophy Warning: Matching character arrays against strongly typed tokens lets you translate raw configuration text files into reliable engine variables.

Milestone 2.3 – Vector Level Construction Layers

Sub-step 2.3.0.1: Unified Physics Databases

Create a `VectorCollisionMap` master struct that holds outer map limits and inner obstacle shapes under a single uniform interface layout container.

Expected Output Example: Map loading readout: "Generated `VectorCollisionMap` tracking 1 outer perimeter and 14 inner structures."

Good Practice Tip: Centralize related shape bounds inside a clean tracking structure to keep level evaluations organized.

Syntax Philosophy Warning: Consolidating coordinate tracking maps under a single structure simplifies spatial lookups during calculation passes.

Sub-step 2.3.0.2: Dual-Use Parsing Pipelines

Build an asset pipeline routine: load an SVG file, dispatch its visual layout elements straight to your rendering loop, and pipe its boundary path codes directly into your physics matrix to generate maps.

Expected Output Example: Asset log trace: "Loaded vector level: 450 rendering nodes split to display, 3 boundary loops sent to physics."

Good Practice Tip: Pull both art assets and boundary data from a single source file to keep your asset management simple and organized.

Syntax Philosophy Warning: Packaging visual files and boundary tracking loops inside an all-in-one format ensures your environment artwork and collision hulls stay perfectly aligned.

Milestone 2.4 – Continuous Frame Updates

Sub-step 2.4.0.1: Isolated Node Index Extraction

Write an indexing helper: given an index key offset (`usize`) inside your `SceneTree` array, extract an exclusive mutable reference to that node without hitting borrow-checker access panics.

Expected Output Example: Selection logging: "Accessing node index: 4 [Tag: 'Enemy_Orc_Elite'] for position transformation updates."

Good Practice Tip: Avoid holding onto long-term mutable references while querying other elements in your index matrix.

Syntax Philosophy Warning: Temporarily borrowing specific data items into stack memory allows you to perform complex spatial mutations without running into borrow checker blocks.

Sub-step 2.4.0.2: Broad-Phase Verification Passes

Wire directional movement keys (WASD) to adjust velocity. Before committing coordinate transformations, query your `VectorCollisionMap` to verify spatial clearance, blocking entities from sliding through walls.

Expected Output Example: Target move position: (45.2, 12.8) -> Collision check clear -> Position committed.

Good Practice Tip: Check your target coordinates against the obstacle map before committing any movement updates to prevent characters from getting stuck inside walls.

Syntax Philosophy Warning: Using a clear double-pass check keeps your physics processing isolated from your core scene properties, making state updates predictable and stable.

PROJECT 03: FT_COMMAND_RUNTIME (THE SCRIPT RUNTIME)

Isolate entity states from execution behaviors using the Command Pattern. Build an evaluation virtual machine loop that processes actions across a pointerless engine matrix.

Turn-in directory: ex03/

Dependencies: std::*

Milestone 3.0 – Logic Encapsulation Enums

Sub-step 3.0.0.1: The Custom Command Variant Blueprint

Create a `ScriptCmd` enum that converts script actions into data payloads: `ShowText(String)`, `SlideTo { id: usize, tx: f32, ty: f32, speed: f32 }`, and `ToggleSwitch(String)`.

Expected Output Example: `ScriptCmd::SlideTo { id: 1, tx: 500.2, ty: 300.1, time: 2.0 }`

Good Practice Tip: Keep your command parameters simple to ensure your bytecode operations evaluate quickly and reliably.

Syntax Philosophy Warning: Using explicit enum payloads allows you to decouple your logic commands from your active runtime data structures.

Sub-step 3.0.0.2: Node Callback Configuration Hooks

Augment your `GameNode` struct with a code callback field: `attached_script: Option<Vec<ScriptCmd>>`. Hardcode a dummy script sequence in memory to test validation loops.

Expected Output Example: Attached interaction script containing 3 discrete execution commands.

Good Practice Tip: Wrap your action triggers in an `Option` container so you can skip calculation checks for inert objects.

Syntax Philosophy Warning: Using an `Option` layout allows you to represent the complete absence of a data value safely without using null pointers.

Milestone 3.1 – Isolated Engine Evaluator Passes

Sub-step 3.1.0.1: The Static Match Evaluator

Write an isolated evaluation function. Pass a simple list of commands to a `match` loop and check if it modifies dummy test states correctly before connecting it to your world data systems.

Expected Output Example: Evaluating command `ShowText("Hello World")` -> Printed text successfully.

Good Practice Tip: Test your command structures using a mock state loop to verify routing logic before connecting it to your master world systems.

Syntax Philosophy Warning: Isolating your evaluation logic in clean match blocks makes adding new script instructions simple and error-free.

Milestone 3.2 – Borrow Split Resolution Loops

Sub-step 3.2.0.1: Resolving Vector Borrow Mismatches

Write an evaluation routine: `fn execute_script(scene: &mut SceneTree, node_idx: usize)`. You will trigger borrow errors if a script tries to mutate an entity while borrowing the master array layout.

Expected Output Example: (Expected Compiler Block) `error[E0502]: cannot borrow '*scene' as mutable because it is also borrowed as immutable`

Good Practice Tip: Explain how the compiler blocks simultaneous mutable and immutable vector reads to your evaluator.

Syntax Philosophy Warning: Rust prevents you from modifying a collection while iterating through its elements to protect against pointer invalidation.

Sub-step 3.2.0.2: Localized Target Script Cloning

Solve borrow conflicts by cloning or shifting targeted script blocks into temporary stack memory frames before triggering array update passes.

Expected Output Example: Extracted script array successfully -> Executed operations across world targets.

Good Practice Tip: Clone the targeted instruction array into a temporary workspace to free up access to the master scene matrix.

Syntax Philosophy Warning: Making a localized copy of your command steps allows you to perform structural mutations safely without running into reference lockouts.

Milestone 3.3 – Global Switches Mapping Databases

Sub-step 3.3.0.1: The Global Flag Registry Matrix

Embed a `HashMap<String, bool>` database container into your `SceneTree` state to track global flags (like quest steps or locked doors).

Expected Output Example: Global Switch State: {"BOSS_DEFEATED": true, "CHEST_OPENED": false}

Good Practice Tip: Track all world state flags in a centralized hash registry to keep progress monitoring clear and straightforward.

Syntax Philosophy Warning: Using a centralized registry keeps global parameters isolated from individual entity structures.

Sub-step 3.3.0.2: Conditional Flow-Jump Checking

Add a new instruction payload: `ScriptCmd::IfSwitch { flag: String, body: Vec<ScriptCmd> }`. Update your interpreter loop to check the hashmap registry before deciding to run or skip nested blocks.

Expected Output Example: Evaluating IfSwitch("CHEST_OPENED") -> Condition evaluated to False -> Nested branch bypassed.

Good Practice Tip: Evaluate condition flags systematically before step-processing nested command branches.

Syntax Philosophy Warning: Branching logic structures evaluate condition keys dynamically, allowing you to build reactive storytelling frameworks safely.

PROJECT 04: FT_VGE_COMPILER (THE COMPILER FRONTEND)

Construct a custom compiler frontend that reads human-readable text files (`.vge`), tokenizes syntax elements, and builds an executable Abstract Syntax Tree.

Turn-in directory: ex04/

Dependencies: logos, chumsky

Milestone 4.0 – Grammatical Token Component Mapping

Sub-step 4.0.0.1: Core Lexical Variant Specifications

Define a `Token` enum explicitly mapping structural parts of speech: Keywords (`If`, `Say`, `Slide`), matching delimiters (`(`, `)`, `{`, `}`), and custom string identifiers.

Expected Output Example: Token::If, Token::Identifier("slide_to"), Token::Float(45.2)

Good Practice Tip: Structure your keyword definitions clearly within a type-safe token lexicon enum.

Syntax Philosophy Warning: Strongly typed enums catch language construction issues early, keeping compiler pipelines predictable.

Milestone 4.1 – Character Stream Token Scanning

Sub-step 4.1.0.1: String Stream Ingestion Pipelines

Use `logos` to build a lexical scanner. Read a raw script string from disk and transform it into a flat token vector array: `Vec<Token>`.

Expected Output Example: Token Stream: [If, LeftParen, Identifier("FLAG"), RightParen, LeftBrace]

Good Practice Tip: Ensure your token definition rules capture complex text sequences like string literals correctly.

Syntax Philosophy Warning: Lexical scanning organizes raw, unformatted text files into structured, sequential word token blocks.

Sub-step 4.1.0.2: Whitespace and Non-Semantic Strip Loops

Configure your scanner macro rules to automatically catch, filter, and discard blank spaces, code indentation tabs, and script comment lines from the output token stream.

Expected Output Example: Whitespace and comment markers are cleanly stripped from the output token stream.

Good Practice Tip: Configure your scanner rules to automatically discard non-semantic elements like whitespace or comments to keep the output token stream clean.

Syntax Philosophy Warning: Removing non-semantic layout elements from the token stream simplifies your core grammar rules.

Milestone 4.2 – Flat Expression Translation Mechanics

Sub-step 4.2.0.1: Linear Slice Matching Utilities

Write an elementary grammar matcher block. Process small sub-slices of your token vector to verify layout sequences without nested logic structures yet.

Expected Output Example: Matching slice: [Identifier("say"), StringLiteral("Hello")] -> Match Valid.

Good Practice Tip: Test your translation rules against short, linear token sequences to verify your core expression formatting.

Syntax Philosophy Warning: Matching token slices against clear structural patterns lets you handle simple syntax rules easily before dealing with nested code scopes.

Sub-step 4.2.0.2: Translation to Instruction Commands

Configure your translator rule: if it matches a `Say` token followed by a valid string literal, map it directly to an executable `ScriptCmd::ShowText` enum data payload.

Expected Output Example: Compiled script block: `ScriptCmd::ShowText("Hello")`

Good Practice Tip: Ensure your translation steps convert simple expression tokens directly into valid engine command payloads.

Syntax Philosophy Warning: Connecting grammatical structures directly to engine commands bridges the gap between text assets and live game logic.

Milestone 4.3 – Context-Free Parsing & Tree Compilation

Sub-step 4.3.0.1: Recursive Combinator Binding Rules

Use `chumsky` to build recursive parsing rules. Define clear structural validations to safely group sequential command lines trapped inside brace layouts (`{ ... }`).

Expected Output Example: Parser Pass: Successfully resolved nested bracket scope structure.

Good Practice Tip: Use parser combinators to enforce clear syntax rules for managing recursive logic blocks.

Syntax Philosophy Warning: Combinators let you build complex parsing rules out of small, easily testable grammar functions.

Sub-step 4.3.0.2: Heap-Wrapped AST Node Building

Ensure that nested branch components are packed using heap allocations (`Box<ScriptCmd>`) to construct your final, executable Abstract Syntax Tree structure.

Expected Output Example: Abstract Syntax Tree compiled successfully with recursive `Box` allocations.

Good Practice Tip: Use heap wrappers (`Box<T>`) to manage recursive branching data safely without running into compiler size constraints.

Syntax Philosophy Warning: Recursive data models require heap pointers because the compiler needs to know the exact stack size of your types at compile time.

Milestone 4.4 – End-to-End Delivery Integration

Sub-step 4.4.0.1: Automated Asset Extraction Pipelines

Write a master loader function that reads an external plain-text file (`level_01.vge`) from disk on initialization and automatically passes it through your tokenizing and parsing pipelines.

Expected Output Example: Loading: `level_01.vge` -> 24 tokens scanned -> 3 AST branches generated.

Good Practice Tip: Design your file loading pipelines to read text assets from disk and pass them directly through your parsing components on initialization.

Syntax Philosophy Warning: Converting disk-based script assets into type-safe code objects on startup protects your runtime loop from sudden file system errors.

Sub-step 4.4.0.2: Launch Verification Passes

Take the compiled `Vec<ScriptCmd>` tree layout, attach it to a node inside your pointerless arena matrix, and run it live within your continuous SVG physics world map!

Expected Output Example: `vge_engine` boot success. Running interactive system loop over continuous SVG space.

Good Practice Tip: Verify that your newly parsed command trees are mounted to their target nodes and evaluate correctly inside your continuous physics frame loop.

Syntax Philosophy Warning: Integrating your parsing frontend directly with your core physics loop closes the loop between text assets and live game mechanics.